Uni Child International School Website and Student Information Management System

Team DevForge — Final Demo and Handoff

Course: DCIT 208 — Software Engineering

Frontend: https://zealous-flower-03357b203.4.azurestaticapps.net

Backend health: https://unichildinternationalschoolbackend-bfg4hwd8fpduavgp.westus3-01.azurewebsites.net/api/health

Problem and Project Motivation

- School information and administrative records need to be accessible and organised.
- Manual processes can make information retrieval and communication more difficult.
- Parents and guardians need clear information about the school and admissions.
- Administrators and teachers need appropriate tools for student-related information.

Project Objectives

- Provide an online platform for Uni Child International School.
- Present school information, academics, admissions, news, events and gallery content.
- Support student information management and role-based access.
- Provide a foundation for managing results, fees and school communications.
- Demonstrate a deployable web application with a separate backend API.

Requirements and Project Scope

- Functional requirements: public visitors browse school information; administrators, teachers and students use designated interfaces; backend routes support authentication, student records, results, fees and content.
- Scope: public website, school information pages, student-management interfaces, backend API, and student/teacher portal interfaces.
- Results, fees and communication features are in scope where implemented and operational.
- Verification note: demonstrate only workflows that work in the current environment. Do not present unverified features as fully operational.

Intended Users

- Administrator — manage student records and relevant school information; perform authorised administrative activities.
- Teacher — access relevant teaching functions and student-related workflows.
- Student — access the student portal and relevant academic information.
- Public visitor — browse school information and admissions-related content.

Main Features

- Public school website and navigation.
- About, academics and admissions pages.
- News, events and gallery pages.
- Student management dashboard and student/teacher portal interfaces.
- Backend API routes for authentication, students, teachers, results, fees, content, events, documents, gallery and enquiries.
- Demonstration note: show only workflows that can be demonstrated successfully.

Technology and Architecture

- Frontend: HTML, CSS and JavaScript.
- Backend: Node.js, Express.js and REST API.
- Data layer: PostgreSQL and Sequelize ORM.
- Hosting: Azure Static Web Apps (frontend) and Azure App Service (backend).
- Delivery: GitHub repository and GitHub Actions deployment workflows.
- Architecture: Browser → Frontend website → Backend REST API → Database.

Example User Workflows

- Workflow 1 — Public visitor: open the website; navigate school information and academics; open admissions or contact information.
- Workflow 2 — Student administration: open administrator dashboard; view student directory; demonstrate registration or record management if operational.
- Workflow 3 — Student/teacher access: open the appropriate portal; authenticate with an authorised test account; show relevant functionality if operational.
- Use authorised test accounts and avoid displaying real student personal information.

Deployment and CI/CD

- Frontend URL: https://zealous-flower-03357b203.4.azurestaticapps.net
- Backend health endpoint returned {"status":"Server is running"} during the deployment check.
- GitHub Actions deployment workflows are configured for the frontend and backend.
- A successful deployment workflow does not prove every application feature works correctly.
- Final verification of API integration and user workflows remains necessary.

Security and Privacy

- Backend authentication uses JSON Web Tokens (JWT).
- Role-based authorisation is implemented through backend middleware.
- Helmet adds HTTP security headers; rate limiting is configured for API requests and login attempts.
- Cross-origin requests are controlled through configured CORS origins.
- Student information and credentials should only be accessed by authorised users.
- Store deployment secrets securely in hosting settings or GitHub Secrets, never in the repository.
- Effectiveness of these controls still requires appropriate verification.

Testing and Quality Assurance

- Evidence obtained: deployed frontend loaded; backend health endpoint responded; frontend and backend deployment workflow evidence was available in GitHub Actions.
- Outstanding checks: authentication and permissions; student registration and updates; results and fees; validation and error handling; mobile layout and accessibility; API integration and database-backed operations.
- Formal end-to-end QA has not yet been completed.
- Automated backend tests have not been established in the supplied project package.
- No unperformed test should be reported as passed.

Sprint Progress and Changes

- Developed the public-facing school website and portal interfaces.
- Established a separate backend API and database integration.
- Configured cloud hosting and deployment workflows.
- Updated frontend API configuration to point to the deployed backend.
- Prepared user, deployment and client handoff documentation.
- Confirm reported activities against actual commits, pull requests and project-board records.

AI Use and Academic Integrity

- AI tools may support brainstorming, explanations, drafting and development assistance.
- Significant AI use must be recorded in project AI-use documentation.
- Team members are responsible for checking generated content and code.
- Team members should be able to explain their contributions and engineering decisions.
- AI usage log and acknowledgements must reflect actual use and genuine verification.
- References: AI_USAGE.md and AI_USAGE_LOG.md.

Known Limitations and Future Improvements

- Formal end-to-end QA remains outstanding.
- Automated backend tests have not been established in the supplied project package.
- Database backup, recovery and rollback procedures require practical verification.
- Client acceptance and feedback have not been documented as confirmed.
- Some application workflows may require further verification or improvement.
- Prioritise future improvements using test results and stakeholder feedback.

Handoff and Maintenance

- Handoff documentation prepared: User Manual; Deployment and Operations Guide; Client Handoff document.
- Next steps: complete and record QA; verify deployed workflows and API integration; capture genuine screenshots; document client feedback where available.
- Confirm the release tag and repository evidence; upload the unlisted demo video; add the actual URL to youtube_links.md.
- Follow the operations guide for configuration, troubleshooting, backup and recovery planning.
- Procedures that have not been tested must remain identified as unverified.

Project Evidence and Release

GitHub repository	https://github.com/niipascal51-tech/unichild-international-school_webproject
Main branch	main
D9 release tag	v1.0-demo
Tagged commit	93012d5 — docs: prepare D9 final demo and handoff
Release tag	https://github.com/niipascal51-tech/unichild-international-school_webproject/tree/v1.0-demo
Deployed frontend	https://zealous-flower-03357b203.4.azurestaticapps.net
Backend health	https://unichildinternationalschoolbackend-bfg4hwd8fpduavgp.westus3-01.azurewebsites.net/api/health
Demo video	Pending: add the unlisted YouTube URL after upload
Project board	Pending: add the team's actual project board URL

Closing

- The project provides an online school website and a foundation for student information management.
- The frontend is deployed and the backend health endpoint responded successfully.
- Complete workflow verification, final handoff evidence and transparent documentation of limitations remain important next steps.
- Team DevForge • DCIT 208 — Software Engineering
- Thank you.